

مشکل (Problem)

یه کلاس شامل یه فیلد **Collection** و یه **getter/setter** ساده برای کار باهاش هست.

راه حل (Solution)

خروجی **getter** رو فقط-خواندنی کن و متدهای جداگانه برای اضافه/حذف عناصر بساز. **setter** رو حذف کن.

مثال از صفحه

قبل:

csharp

```
public class Course
{
    public List<string> Students { get; set; }
}

// استفاده خطرناک:
// کل لیست رو عوض میکنه!
course.Students = new List<string>();
// بدون اطلاع
course.Students.Add("Ali");
// همه رو پاک میکنه!
course.Students.Clear();
```

بعد:

csharp

```
public class Course
{
    private List<string> students = new List<string>();

    public IReadOnlyList<string> Students => students;

    public void AddStudent(string name) => students.Add(name);
    public void RemoveStudent(string name) => students.Remove(name);
}
```

چرا ریفکتور کنیم؟ (Why Refactor)

دلیل	توضیح
getter خود Collection رو برنگردونه	وگرنه کلاینت می‌تونه بدون اطلاع کلاس مالک، محتوا رو تغییر بده
ساختار داخلی لو میره	getter نباید داده‌های زیادی از ساختار داخلی رو فاش کنه
setter نباید وجود داشته باشه	به جاش عملیات اضافه/حذف باشه تا مالک کنترل داشته باشه
کپسوله‌سازی مناسب	وابستگی بین کلاس مالک و کد کلاینت رو کاهش میده

مزایا (Benefits)

مزیت	توضیح
جلوگیری از تغییر تصادفی	getter یه کپی فقط-خواندنی برمی‌گردونه
محدود کردن دسترسی	می‌تونی متدهای ناخواسته (مثل Clear) رو مخفی کنی
کنترل اضافه/حذف	مالک می‌تونه اعتبارسنجی کنه

How to Refactor (چطور انجام بدیم)

قدم	توضیح
۱	متدهای اضافه/حذف برای عناصر Collection بساز
۲	به فیلد Collection یه مقدار اولیه خالی بده
۳	جاهایی که setter صدا زده میشه رو با متدهای اضافه/حذف جایگزین کن

توضیح	قدم
می‌تونی اسم setter رو به Replace تغییر بدی (اگه واقعاً نیازه)	۴
همه جاهایی که بعد از Collection getter رو تغییر میدن رو با متدهای جدید جایگزین کن	۵
getter رو طوری تغییر بده که نمایش فقط-خواندنی برگردونه	۶
کد کلاینت رو بررسی کن — شاید منطقش بهتره توی خود کلاس Collection باشه	۷

تمرین

این کد رو با Encapsulate Collection اصلاح کن:

csharp

```
public class Team
{
    public List<string> Members { get; set; }
}

// استفاده فعلی (خطرناک):
var team = new Team();
team.Members = new List<string>();
team.Members.Add("Ali");
team.Members.Add("Sara");
team.Members.Clear(); // همه رو پاک کرد بدون اطلاع Team!
```

راهنمایی:

- Members رو private کن
- `ReadOnlyList<string>` برای `getter`
- متدهای `AddMember` و `RemoveMember` بساز

بفرست تا بررسی کنیم! 

خودت حلش کن

کد اصلاح شده با Encapsulate Collection

csharp

```

public class Team
{
    ;()private List<string> members = new

;public IReadOnlyList<string> Members => members

    public void AddMember(string name)
    {
        ;members.Add(name)
    }

    public void RemoveMember(string name)
    {
        ;members.Remove(name)
    }

    // استفاده جدید:
    ;()var team = new Team
    ;team.AddMember("Ali")
    ;team.AddMember("Sara")
    ;()team.Members.Clear // ❌ خطای کامپایل! IReadOnlyList این متد رو نداره

```

مقایسه

نسخه اصلاح	کد اصلی
 غیرممکن ❌	 ممکن بود ()Members.Clear
 غیرممکن ❌	جایگزینی کل لیست ...team.Members = new

نسخه اصلاح	کد اصلی
✓ <ReadOnlyList<string	<List<string
✓ RemoveMember / AddMember با متدهای	بدون کنترل
	کنترل اضافه/حذف

Encapsulate Collection تموم شد!

حالا Data Class کامل شد:

وضعیت	درمان
✓	Encapsulate Field
✓	Encapsulate Collection
✓ (قبلاً)	Move Method
✗ جدید	Remove Setting Method
✗ جدید	Hide Method

دو تا جدید داریم. بریم Remove Setting Method؟ متنش رو بفرست!

→ SOON